



innovarsity
learn to innovate. lead to succeed.



NIELIT

skillbank

Integrate Experience Knowledge & skill



NumPy

PYTHON PROGRAMMING

PRESENTATION BY SRINIVASA SAI CHAVA

MODULE 1 OF ESSENTIALS OF AI

ABOUT ME

- M.S. Artificial Intelligence student at Boston University



**BOSTON
UNIVERSITY**

- Building production-focused AI systems across ML/DL, NLP, Computer Vision, and RAG

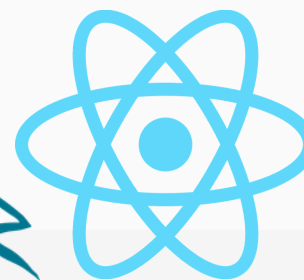


**INDIAN INSTITUTE OF
INFORMATION TECHNOLOGY
DHARWAD**

- Delivering end-to-end full-stack AI deployment from research to production



- Focused on practical impact in accessibility, trust, and social good



<https://github.com/essentials-of-artificial-intelligence>

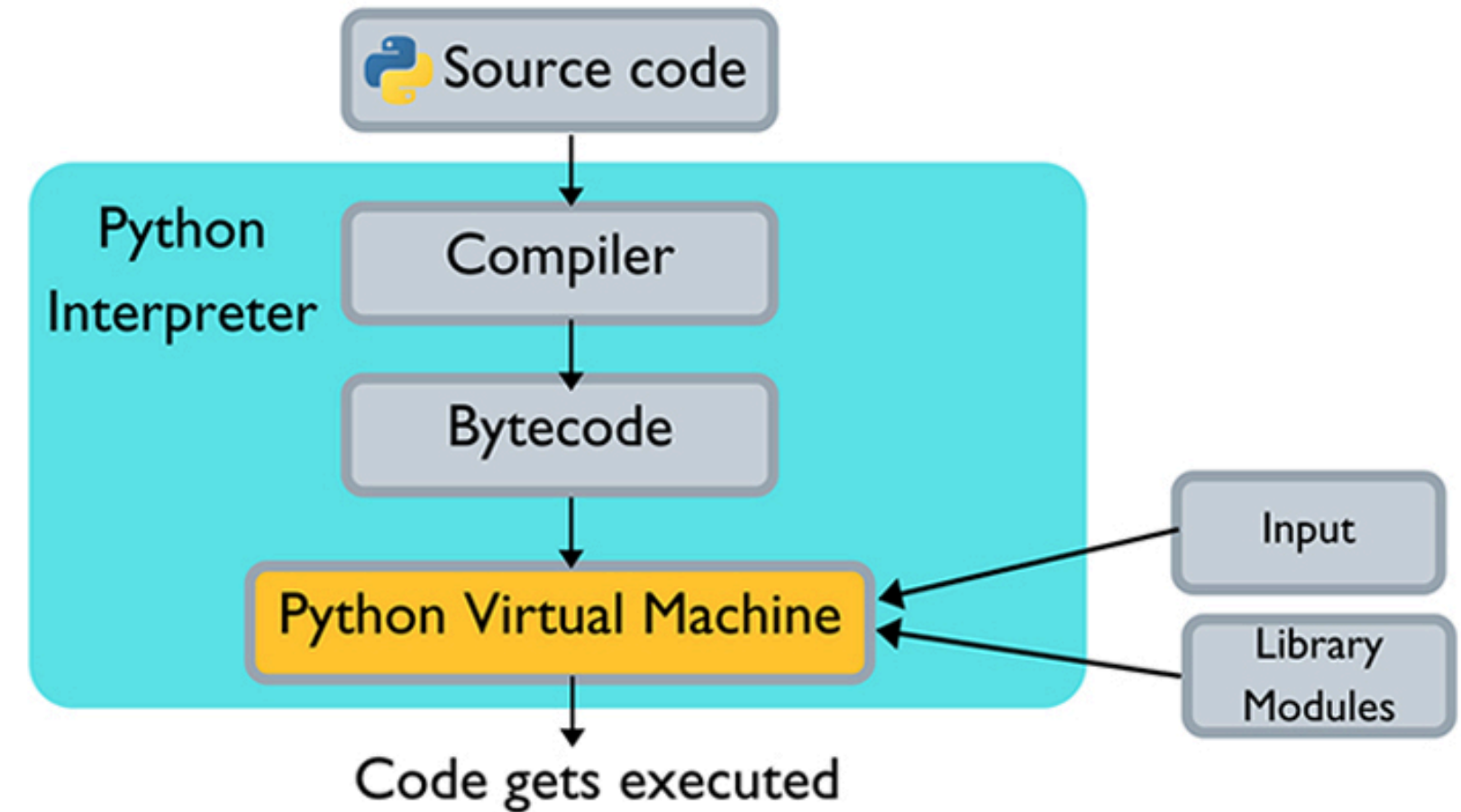
GITHUB ORGANIZATION



<https://github.com/essentials-of-artificial-intelligence>

HOW PYTHON ACTUALLY RUNS

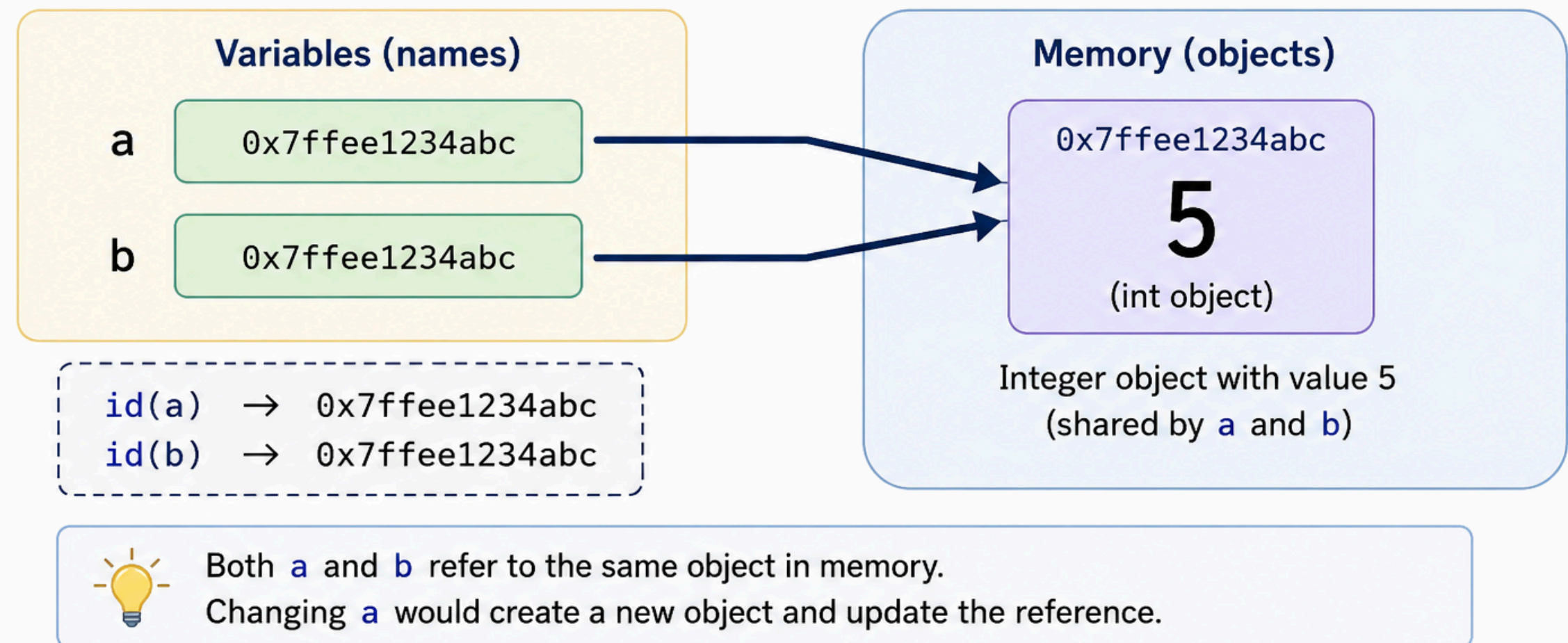
- Python is interpreted but uses a compiler step first (compiles to .pyc bytecode).
- The bytecode runs on the Python Virtual Machine (CPython).
- Why it matters for AI: Python acts as the "glue" that triggers high-speed C/C++ binaries in libraries like NumPy and TensorFlow.



Source Code --> Compiler --> Bytecode --> Virtual Machine.

VARIABLES ARE LABELS, NOT BUCKETS

- In C, a variable is a memory location. In Python, a variable is a pointer to an object in memory.
- Everything is an object (even integers).
- Use the `id()` function to see the memory address of an object.



PRIMITIVE AND COLLECTIONS

DATATYPES



Numeric

int, float, complex

Numeric: `int`, `float`, `complex`
(crucial for advanced math).

```
x = 10      # int
y = 3.14    # float
z = 2 + 3j  # complex
```



Text

`str`

Text: `str`
(Strings are immutable sequences).

```
s = "Hello, Python!"
type(s) # str
```

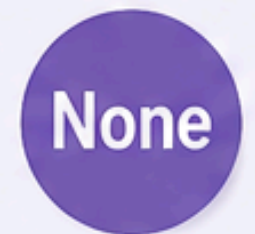


Boolean

`bool`

Boolean: `bool`
(True / False).

```
a = True
b = False
type(a) # bool
```



NoneType

`None`

NoneType: Represents the
absence of a value.

```
n = None
type(n) # NoneType
```



Key Takeaway

Every value in Python is an object of one of these built-in types.

Bringing Static Typing to Python

- Python is dynamically typed, but Type Hints improve IDE auto-completion and catch errors before runtime.
- Essential for large AI codebases (e.g., autograding systems or large data pipelines).
- Syntax:

```
def process_image(pixels: int) -> bool:
```

Why Type Hints?



Better IDE Support

Enables auto-completion, inline documentation, and smarter refactoring.



Catch Errors Early

Static type checkers (e.g., mypy) find type mismatches before you run your code.



Improve Code Quality

Makes code easier to read, maintain, and scale in large projects.

Example

```
from typing import List, Tuple

def process_image(pixels: List[int], threshold: float) -> bool:
    # Returns True if the image has enough bright pixels
    ...

# pixels is a list of integers
# threshold is a float
# returns a boolean
```



Key Takeaway: Type hints don't change how Python runs — they add metadata for tools and humans. They make your code safer, smarter, and easier to collaborate on.

Beyond Basic Math



1. Floor Division (//) vs. True Division (/)

Floor Division (//) returns the integer quotient (rounds down). True Division (/) returns a float quotient.

Expression	True Division (/)	Floor Division (//)
7 / 2	3.5	3
9 / 4	2.25	2
-7 / 2	-3.5	-4
7 / -2	-3.5	-4



2. Modulo (%) for cyclical logic

Returns the remainder after division. Perfect for repeated or cyclical tasks—like printing loss every 10 epochs.

Example

```
for epoch in range(1, 31):  
    train_one_epoch()  
    if epoch % 10 == 0:  
        print(f"Epoch {epoch}: loss = {get_loss():.4f}")
```



Prints loss at epochs 10, 20, 30...



3. The Walrus Operator (:=)

Assigns and returns a variable in the same expression. Great for condensing while loops.

Without Walrus

```
line = f.readline()  
while line:  
    process(line)  
    line = f.readline()
```

VS.

With Walrus (:=)

```
while (line := f.readline()):  
    process(line)
```



Why It Matters: These operators help write shorter, cleaner, and more expressive code—crucial for high-performance AI development.



Implicit Booleans

In Python, many values are treated as **True** or **False** in boolean contexts (like **if** statements).



1. You don't always need

```
if len(data) > 0:
```

Python can evaluate the object itself as **True** or **False**.



2. Falsy values

```
0, 0.0, "", [], {}, None
```

These values are considered **False**.



3. Truthy values

```
Everything else
```

Non-zero numbers, non-empty collections, non-empty strings, custom objects, etc.



4. "Pythonic" check

```
if not data:
```

instead of `if len(data) == 0:`

Examples

Expression	Evaluates to
<code>bool(0)</code>	False
<code>bool(0.0)</code>	False
<code>bool("")</code>	False
<code>bool([])</code>	False
<code>bool({})</code>	False
<code>bool(None)</code>	False
<code>bool(42)</code>	True
<code>bool(3.14)</code>	True
<code>bool("hello")</code>	True
<code>bool([1, 2])</code>	True
<code>bool({"a": 1})</code>	True



Think of it as:

If Python can see something in it, it's **True**.
If it's "empty" or "nothing", it's **False**.



Real-world tip

Use **if not data:** to handle empty lists, missing values, or optional inputs cleanly and efficiently.



Key Takeaway:

Python's implicit booleans make your code shorter, cleaner, and more expressive. Embrace **"if not data:"** for elegant and Pythonic logic.

Controlling Execution Flow

1
2
3

1. Evaluates top-to-bottom.

Stops at the first **True** condition.

```
if score > 90:  
    grade = "A"  
elif score > 50:  
    grade = "B"  
else:  
    grade = "F"
```

Python checks each condition in order and runs the first one that is **True**.



2. Ternary Operator for one-liners

A compact way to write simple if/else statements.

```
status = "Pass" if score > 50 else "Fail"
```

Result if condition
is **True**

Condition

Result if condition
is **False**

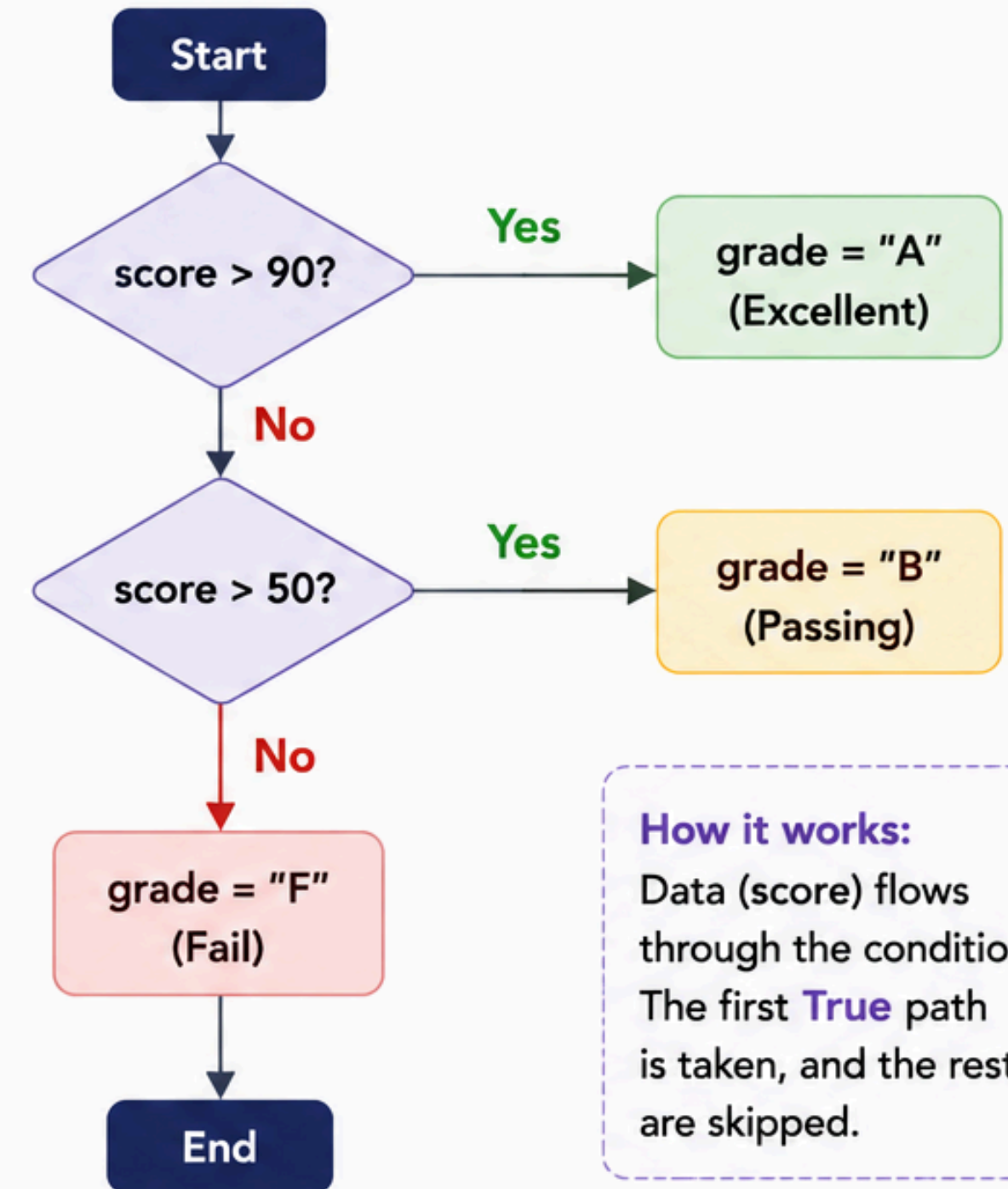


Think of it as a decision tree:

Your program follows the path of the first True condition.

Decision Flow (Example)

Routing a score to a grade



How it works:

Data (score) flows through the conditions. The first **True** path is taken, and the rest are skipped.

The Modern switch Statement



1. Uses match and case.

Introduced in Python 3.10, it provides a clean and expressive way to handle multiple conditions.



2. Much more powerful than C-style switches; can unpack data structures.

Match on values, types, sequences, mappings, and even nested patterns.



3. Ideal for routing different types of AI inputs

Easily dispatch logic based on the kind of data received (e.g., text vs. image vs. audio).

>_ Example

```
def handle_input(data):  
    match data:  
        case {"type": "text", "content": text}:  
            # Handle text input  
            return process_text(text)  
        case {"type": "image", "pixels": pixels}:  
            # Handle image input  
            return process_image(pixels)  
        case {"type": "audio", "samples": samples}:  
            # Handle audio input  
            return process_audio(samples)  
        case _:  
            # Fallback for unknown input types  
            return raise ValueError("Unsupported input type")
```



Why it matters:

Pattern matching reduces boilerplate code, improves readability, and makes complex routing logic easy to extend—perfect for AI pipelines that handle diverse data formats.

C-style switch (Limited)

```
switch(type) {  
    case 1: ... break;  
    case 2: ... break;  
    // No easy destructuring  
}
```

VS.

Python match (Powerful)

- ✓ Match on values, types, and structures
- ✓ Unpack data directly in patterns
- ✓ Cleaner, safer, and more expressive

Definite Iteration



1 Iterates directly over the items of a sequence (list, string, tuple).

The for loop takes each item from the iterable, one at a time.

```
fruits = ["apple", "banana", "cherry"] # apple
for fruit in fruits:                  # banana
    print(fruit)                      # cherry
```



2 range(start, stop, step) generates a sequence of numbers on the fly.

Perfect for counting, indexing, and looping a specific number of times.

```
for i in range(2, 10, 2):
    print(i) # Output: 2 4 6 8
```

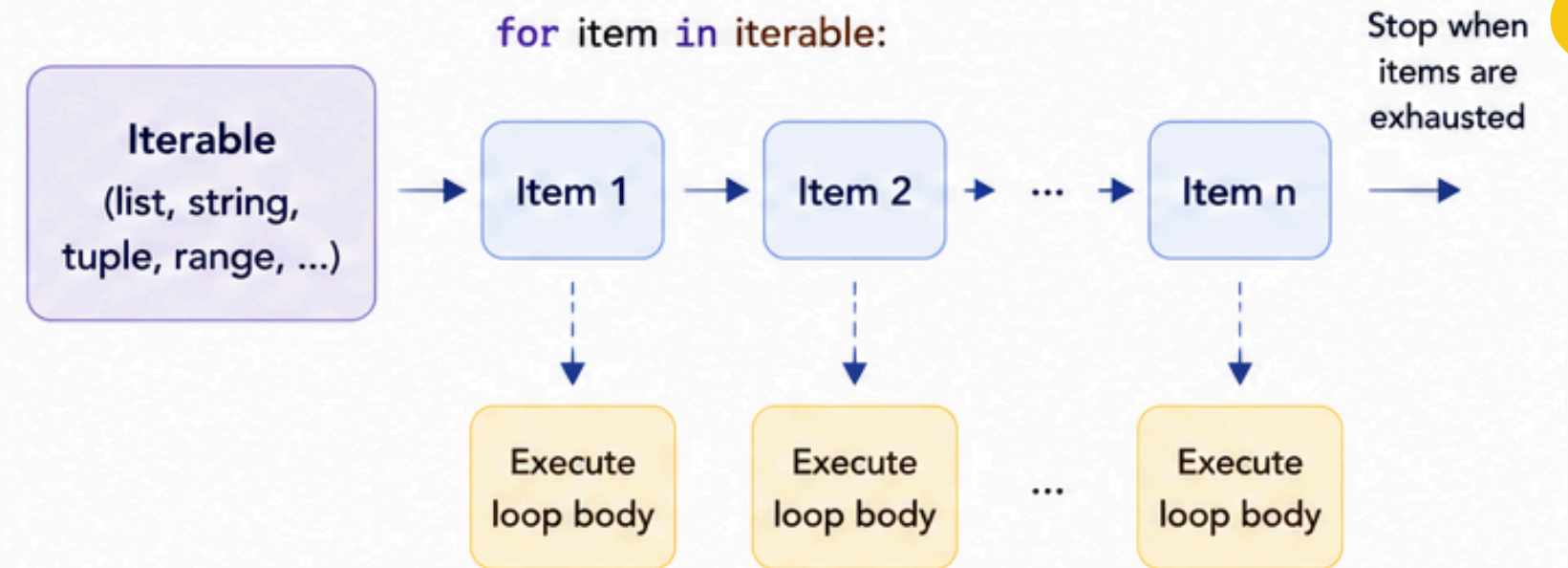


3 Memory efficient: range() doesn't store all numbers in RAM at once.

It generates numbers lazily, which saves memory—especially for large ranges.

```
r = range(1, 1_000_000_000) # range(1, 1000000000)
print(r)                   # Not a huge list in memory!
```

How the for Loop Works



Common Iterables

Iterable	Example	Iteration Yields
 List	numbers = [10, 20, 30]	10, 20, 30
 String	text = "AI"	'A', 'I'
 Tuple	coords = (1, 2, 3)	1, 2, 3
 Range	range(5)	0, 1, 2, 3, 4



Key Takeaway: The for loop is Python's way of performing definite (countable) iteration over iterables. Use range() for numeric loops—it's fast, clean, and memory efficient.



enumerate() and zip()



- 1 **enumerate():** Get the index AND the value simultaneously without manually tracking a counter.

Cleaner, more Pythonic, and less error-prone.

Example

```
fruits = ["apple", "banana", "cherry"]
print("With enumerate():")
for idx, fruit in enumerate(fruits):
    print(f"{idx}: {fruit}")

# Output:
0: apple
1: banana
2: cherry
```

How enumerate() works

Index	Value
0	apple
1	banana
2	cherry



No need to initialize a counter variable manually!



- 2 **zip():** Loop through two or more lists at the same time in parallel.

Stops when the shortest iterable is exhausted.

- 3 **Essential for processing paired data (e.g., features and labels).**

Perfect for ML datasets, metrics, and more.

Example

```
features = [0.2, 0.5, 0.8, 1.0]
labels = [0, 1, 0, 1]
print("Using zip():")
for x, y in zip(features, labels):
    print(f"feature: {x} -> label: {y}")

# Output:
feature: 0.2 -> label: 0
feature: 0.5 -> label: 1
feature: 0.8 -> label: 0
feature: 1.0 -> label: 1
```

How zip() works

features	labels	Paired Output
0.2	0	(0.2, 0)
0.5	1	(0.5, 1)
0.8	0	(0.8, 0)
1.0	1	(1.0, 1)



Great for iterating over related data together!



Key Takeaway: Use `enumerate()` to get index and value together. Use `zip()` to iterate over multiple iterables in parallel. Both make your code cleaner, more efficient, and more Pythonic!



Indefinite Iteration



1 Runs as long as a condition is True.

The code inside the loop repeats while the condition evaluates to True.

```
while condition:
    # loop body
    # condition is checked again
```



2 Dangerous if you don't update the condition (infinite loops).

If the condition never becomes **False**, the loop will never stop!

```
while True:
    pass # This will run forever!
```



3 Used in AI for training loops that run until a specific convergence threshold is met.

Ideal when you don't know the exact number of iterations in advance.

</> Example

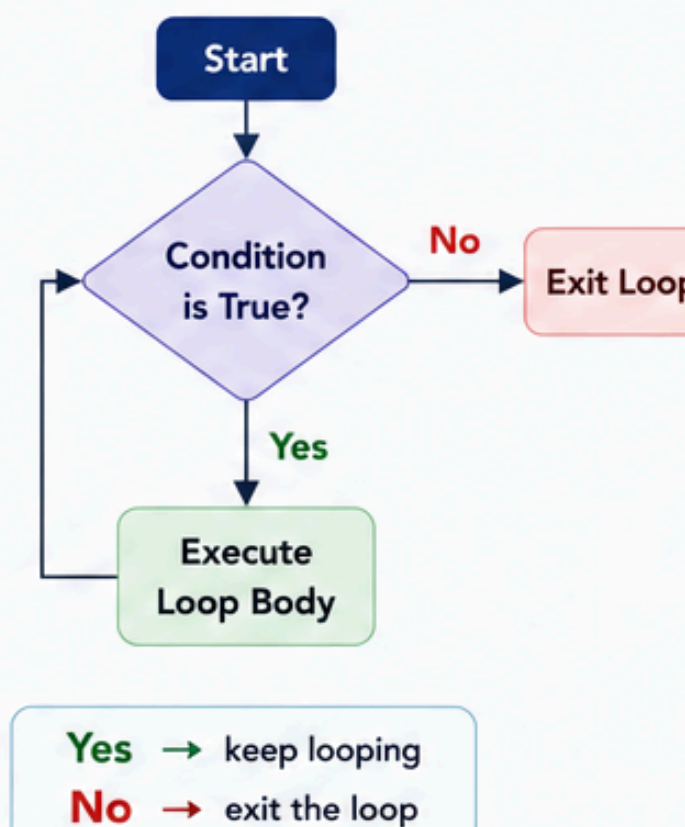
```
epsilon = 0.01
loss = 1.0
epoch = 0

while loss > epsilon:
    loss = compute_loss()
    epoch += 1
    print(f"Epoch {epoch}: loss = {loss:.4f}")

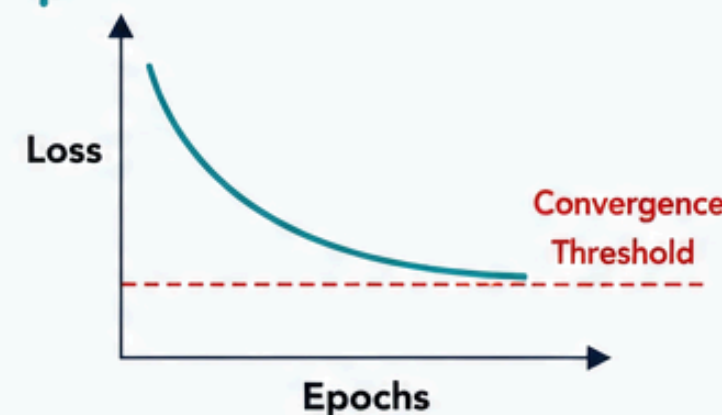
# Loop continues
# until loss is small enough
# (converges)

print("Training converged!")
```

⚙️ How the while Loop Works



AI Use Case: Training Loop



The model trains repeatedly (loop) and updates its parameters until the loss falls below a threshold (converges) or another stopping condition is met.



Goal: Achieve the best model performance efficiently.

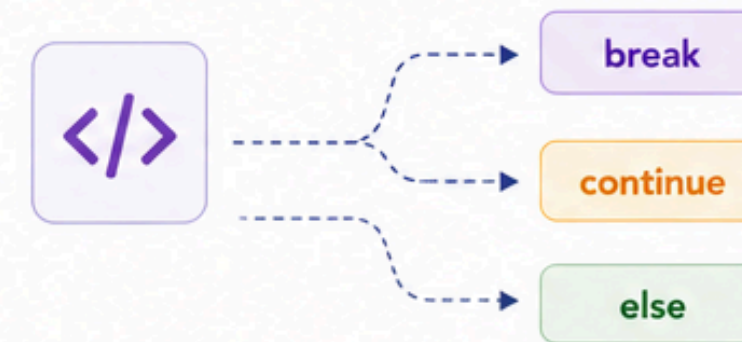


Key Takeaway:

Use **while** loops when the number of iterations is unknown and depends on a changing condition. Always ensure the condition will eventually become **False** to avoid infinite loops!



Modifying Loop Behavior



These keywords give you fine-grained control over loop execution.

1



break

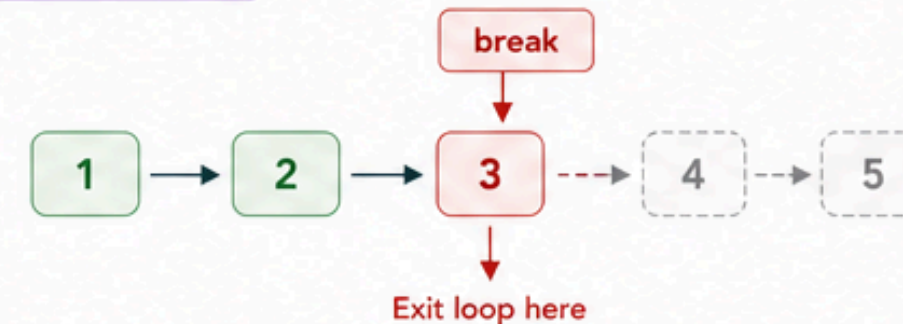
Exits the loop entirely.

Example

```
for i in range(1, 6):
    if i == 3:
        break # Exit the loop
    print(i)
```

Output: 1 2

How it works



💡 Once **break** is encountered, Python exits the loop immediately.

2



continue

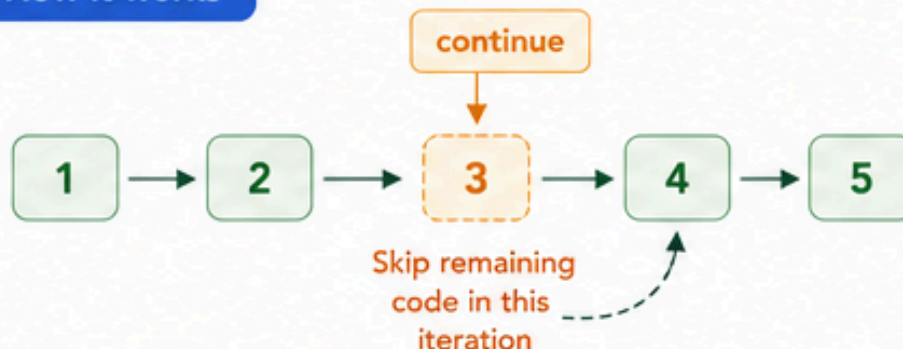
Skips the rest of the current iteration and moves to the next.

Example

```
for i in range(1, 6):
    if i == 3:
        continue # Skip 3
    print(i)
```

Output: 1 2 4 5

How it works



▶▶ It doesn't exit the loop—just skips to the next iteration.

3



for ... else

The else block runs only if the loop completes without hitting a break.

Example

```
for i in range(1, 6):
    if i == 5:
        break
    print(i)
else:
    print("Loop completed without break!")
```

Case A: No break

```
for i in range(1, 4):
    print(i)
else:
    print("Completed!")
```

Output: 1 2 3 Completed!

Case B: break occurs

```
for i in range(1, 4):
    if i == 2:
        break
    print(i)
else:
    print("Completed!")
```

Output: 1

★ Else runs only when the loop finishes naturally.



Key Takeaway

- **break**: Stop the loop early.
- **continue**: Skip to the next iteration.
- **else**: Runs only if the loop wasn't interrupted by a break.



Mastering these gives you precise control over your loops—crucial for writing efficient and correct AI training code.



Preparing for Data Structures

Great job today! You've built the foundation.
Now we level up. 🚀



1 Today: Syntax, memory, and flow.

You've learned how Python thinks and executes code. These are the building blocks of every program—including AI.



2 Tomorrow: Lists, Dictionaries, and Sets (the engines of data preprocessing).

We'll explore Python's core data structures that power real-world AI workflows.



3 Hands-on Lab: Jump into Jupyter/Colab to execute these concepts.

Practice makes it stick. Run the code, experiment, and make it yours!



Keep Going!

Consistency today, mastery tomorrow.

You're building the future—one line of code at a time.

Your Python Journey



Day 1 (Today)

Python Basics

Syntax • Memory • Flow • Control



Day 2 (Tomorrow)

Data Structures

Lists • Dictionaries • Sets



What's Next?

Functions, Modules, and Beyond!

Building powerful Python programs for AI.



See you tomorrow! 😊